



维基百科Android App测试

小组成员：彭博 赵浩然 田红森 吴予哲



目录

01项目概况与综合覆盖率

单元测试1326项全部通过,
综合覆盖率提升至44%

02各包覆盖率排行与提升

Top25包分析,Instrumented
测试带来最大79%提升

03测试执行与成果展示

GUI测试41个文件,手动测
试22项全部通过

04根因分析与改进建议

全局单例问题根因,短期/中
期/中期/长期改进路线图



01

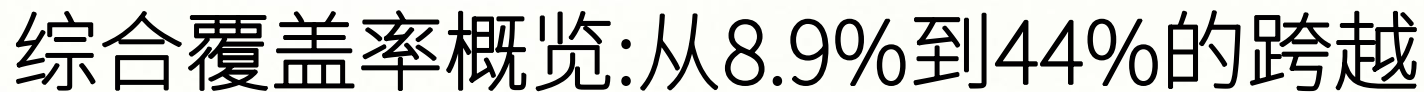
项目概况与综合覆盖率

150个单元测试文件 · 1326个测试用例

• 100%通过率



大连理工大学



8.9%

行覆盖率

28.391/319267行



8.9%

行覆盖率

28.391/319.267行



- 3.8%

分支覆盖率

28.391/319.267行

综合覆盖率(合并)

44.0%

行覆盖率

约27,500/62,673行



44.0%

分覆盖率

约27,500/62.673行



- 23.5%

分支覆盖率

约27,500/62,673行

新增测试增量贡献

+9.5%

行覆盖率

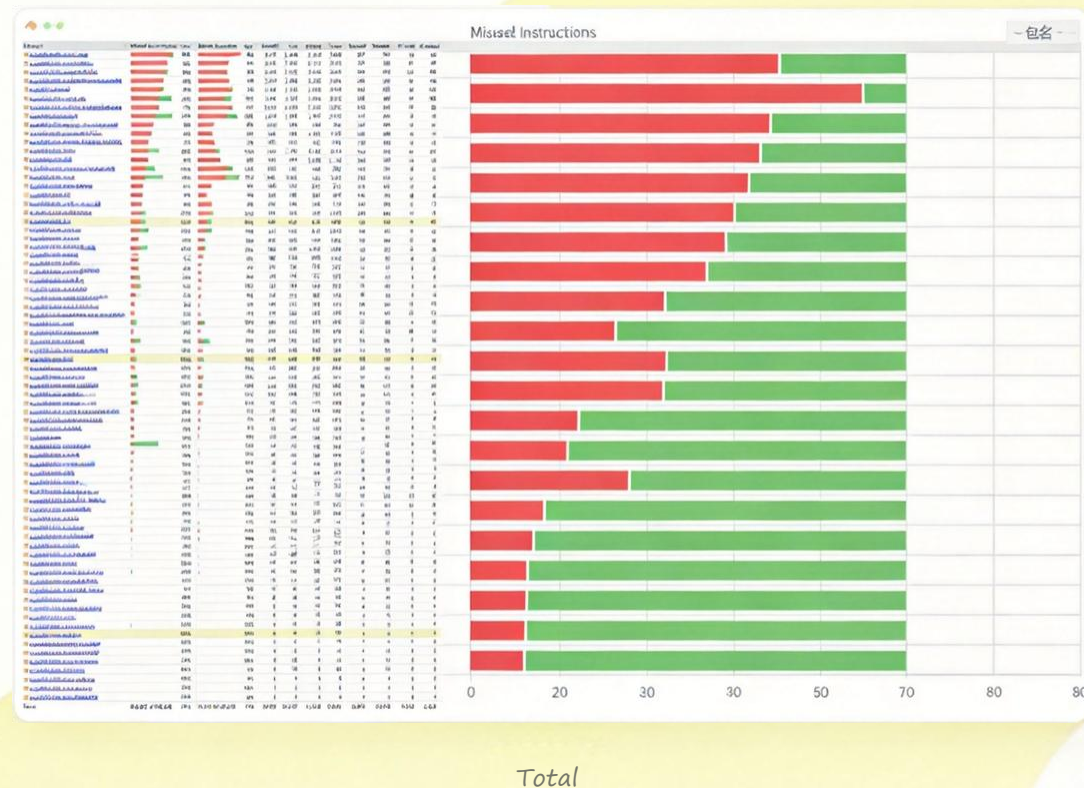
5,900行去重后增量I



+6.5%

行覆盖率

图2.JaCoco覆盖率报告





新增测试逐类覆盖率明细



五者去重后增量贡献:+9.5%行覆盖率,5,900行

02

各包综合覆盖率排行

按代码量排序

Instrumented测试带来的显著提升





Top25包综合覆盖率排行(上)

staticdata 85.9%

静态数据模块 2.457%

page 65.0%

页面核心逻辑 2,400%

settings 54.2%

设置逻辑 1.3.0%

视觉模件 43.5%

views 2.414%

search 44.4%

search 2,086%

搜索模块 48.5%

readinglist.db 2.097%

阅读列表数据库

readinglist 33.0%

阅读列表功能 2,848%

suggestededits 9.2%

建议编辑 2,606%

建议编辑 9.8%

activitytab 9.8%

活动标签 2,460%

talk 11.2%

讨论模块 1,797%

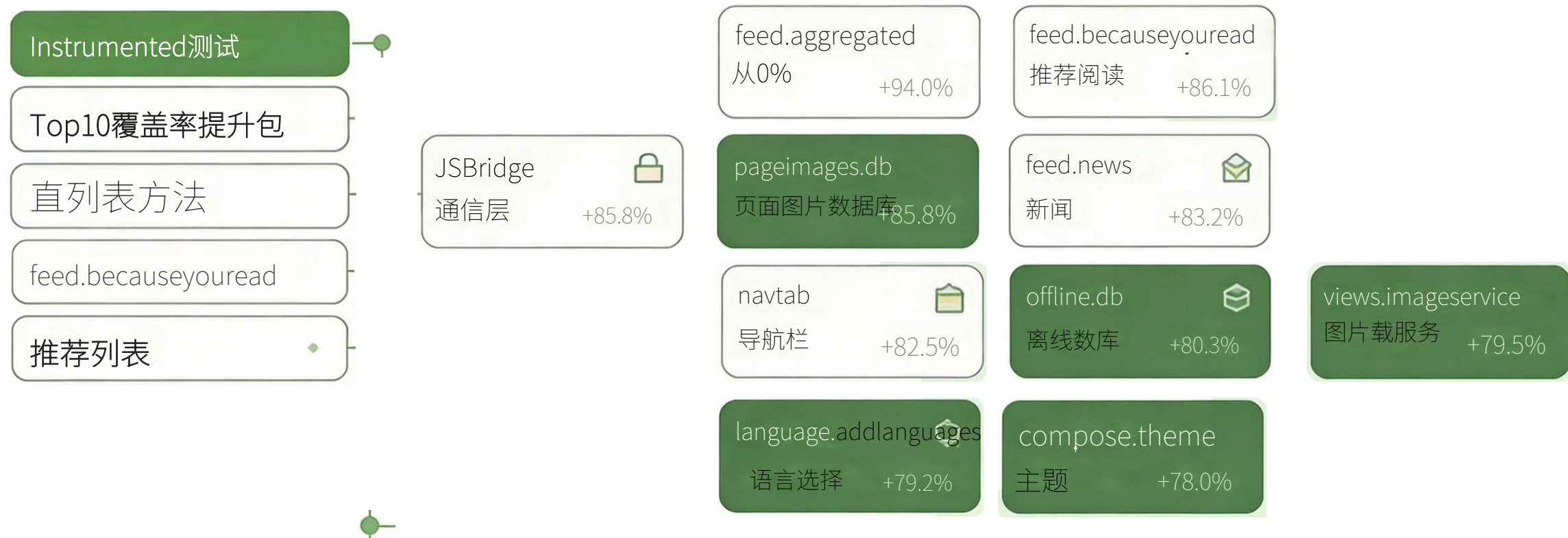
推荐列表 37.4%

notifications 1,070%





Instrumented测试带来的最大覆盖率提升



海纳百川自强不息
厚体笃学知新合一

▲ Instrumented测试覆盖了大量单元测试无法触及的UI与GUI代码路径

03

测试执行与成果展示

单元测试1326项 · GUI测试41个文件

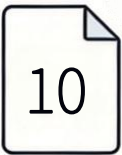
· 手动测试22项 · 全部通过





Instrumented GUI测试概况:41个测试文件全覆盖

tests/articles



文章操作



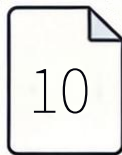
分区



标签页



编辑



5项测试



打开/滚动



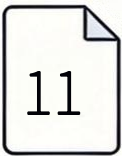
目录 保存



返回



搜索功能



tests/settings



主题



tests/explorefeed信息流卡片与导航



中英文搜索/空课/联想建议



tests/



根目录



离线



引导



搜索



底部导航



测试



完整用户旅程搜索→阅读→保存流流程



集成测试套件:13个测试,全部通过



单元测试执行结果:1326项测试,100%通过率



1326

• 测试方法数

0

失效数

0跳过

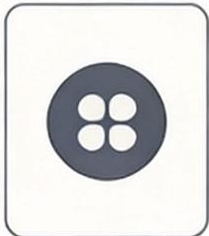
全部通过

2分42秒

2026年6月17日

单元测试文件统计

563arch	80个方法	80files	方送	充制测试
search	10个文件		文件	~80个文件
analytics	9个文件		文件	85个文法
util	12个文件		文件	~60个方法
restbase	10个文件		文件	-25个方法



覆盖率提升对比:新增测试的增量贡献

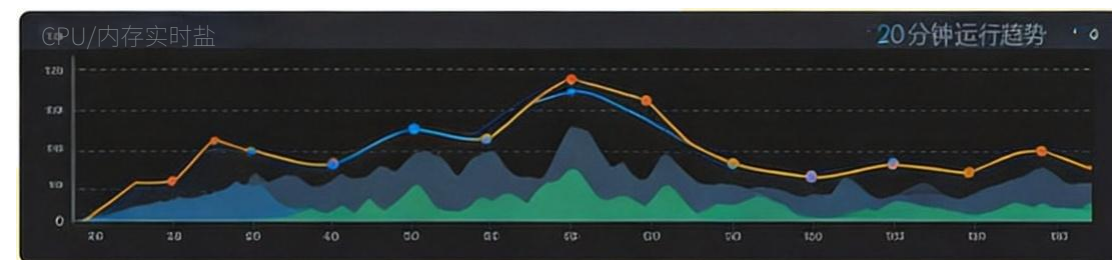
dataclient.watch	提升幅度:个91.7%
之前:0%-> 现在:91.7%	91.7%
Watch数据模型与响 应类全覆盖	Watch数据模型与响 应类全覆盖

dataclient.growthtasks	提升幅度:个88.9%
之前:0% —→ 现在:88.9%	88.9%
增长任务图像建议与用 户影响测试	增长任务图像建议 与用户影响测试

dataclient.restbase	提升幅度:个87.7%
之前:0% —→ 现在:87.7%	现在: 87.7%
差异响应、页面浏览、 编辑计数全覆盖	差异响应、页面浏览、 编辑计数全覆盖

dataclient.donate	提升幅度:个60.9%
之前:0% —→ 现在:60.9%	60.9%
捐赠配置、支付方式、 活动管理	捐赠配置、支付方式、 活动管理

新增测试共覆盖7个关键数据包,平均提升54.7%



整体内存水位安全可控,线程数与Native缓存存在优化空间

整体内存水位安全可控,线程数与Native缓存存在优化空间



04



低覆盖率根因 与改进建议

全局单例泛滥是核心障碍 · 短期/中期/长期三阶段改进路线图





根因分析:全局单例泛滥是测试的核心障碍

WikipediaApp.instance

全项目约300+处引用 · 几乎所有包受影响



AppDatabase.instance

appDaFactory.get()

okhttp 19% · donate 61% · activitytab 1%



ServiceFactory.get(

donate 61% · activitytab 1%

Prefs.xxx

analytics 65% · donate 61% · auth 16%

项目缺少统一的依赖注入框架,Hilt/Dagger虽有使用但不全面。

大量类直接引用全局单例而非通过构造函数注入依赖,导致几乎每个类都与运行时环境强耦合,无法

独立测试。



核心问题:依赖注入不彻底

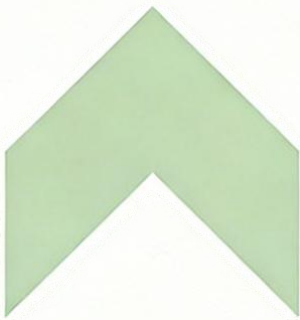
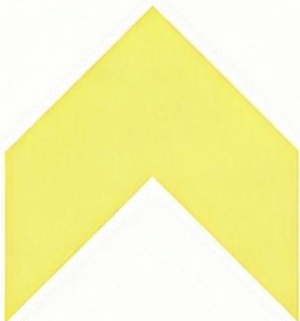
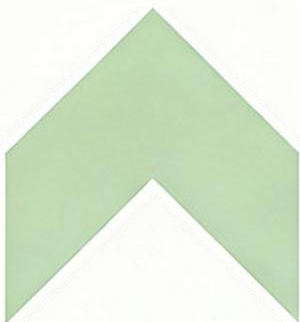


→测试隔离困难→覆盖率提升受阻





无法覆盖的代码归类:约26,000行待攻克



「ComposeU函数」 约26,000行需Compose Testing+Robolectric · 改善难度大
「Activity/Fragment生命周期」 约15,000行需Robolectric/Espresso · 需运行时环境
「Activity/Fragment生命周期」 约15,000行需Robolectric/Espresso · 需运行时环境
「Room 译器生成代码」 约2,900行框架自动生成代码 · 应从覆盖率 除
「Room编译器生成代码」 约2,900行框架自动生成代码 · 应从覆盖率排除
「Paging3分页源」 约2,000行需Paging Testing+Room+网络mock
「OkHttp内部合代码」 约,900行需Paging Testing+Room+网络mock
「OkHttp内部耦合代码」 约900行可重构后测试 · 改善可行性高
「OkHttp内部耦合代码」 约900行可重构后测试 · 改善可行性高
「网络请求lambda与全局单例副作用」 约480行可重构为可注入 · 改善可行性高

优先攻克可改善项,框架生成代码直接排除



改进建议:短期与中期措施

短期改进(预计提升5-10%)

PO为未测试的事件类添加JSON反序列化测试

目标:analytics+3~5%,notifications +5~10%

PO为GrowthUserImpact添加时间注入

目标:growthtasks +10%

P1为ThreadItem添加JSON反序列化测试

目标:growthtasks +10%

P1为ThreadItem添加JSON反序列化测试

目标:discussiontools +25%

P1扩大OfflineCacheInterceptor的mock覆盖

目标:okhttp+3~5%

中期改进(预计提升15-25%)

PO OfflineCacheInterceptor的mock覆盖

中期改进 (15%~25%)

PO CampaignCollection引入协程调度器注入

目标:oknate+30~40%

PO CampaignCollection引入协程调度器注入

目标:donate+20~25%

P1 AccountUtil重构为可注入类

目标:auth+30~50%

P1 ActivityTabViewModel编写Robolectric测试

目标:activitytab+5~10%





长期改进与四阶段路线图





感谢聆听

维基百科Android App测试报告